

Bedienungsanleitung: A.R.T.E. System & Module

Lead Architect: Emanuel Schaaf | Open Origin Architecture

Diese Anleitung beschreibt den korrekten Start und die Initialisierung des A.R.T.E. Systems, das Laden der dazugehörigen Module sowie die grundlegenden Richtlinien für Neuentwicklungen und die Nutzung des App-Stores.

1. Systemstart (Zwingende Reihenfolge)

Das A.R.T.E. System erfordert eine strikte Boot-Reihenfolge. Das Kernsystem muss immer als Erstes geladen werden.

1. Navigiere auf dem Computer in den Ordner `./System`.
 2. Öffne die darin enthaltene Datei `ARTE_System_Core.html` in einem modernen Webbrowser (z. B. durch einen einfachen Doppelklick auf die Datei).
 3. Warte, bis das "Open Origin - OS Control Room" Dashboard vollständig geladen ist und der Status "Bereit" anzeigt.
-

2. Module laden

Nachdem der Systemkern läuft, müssen die gewünschten Module aus dem Ordner `/Module` in das System injiziert werden. Dafür gibt es zwei Möglichkeiten:

Methode A: Drag & Drop (Empfohlen)

1. Öffne den Datei-Explorer und navigiere zum Ordner `/Module`.
2. Markiere die gewünschten HTML-Modul-Dateien.
3. Ziehe die Dateien mit der Maus (Drag & Drop) direkt in die gestrichelte "Drop-Zone" im ARTE Core Dashboard (gekennzeichnet mit "Module hierher ziehen").

Methode B: Manueller Upload

1. Klicke auf die "Drop-Zone" im ARTE Core Dashboard.
2. Es öffnet sich ein Dateiauswahldialog.
3. Navigiere zum Ordner `/Module`, wähle die gewünschten HTML-Dateien aus und bestätige den Upload.

 **Hinweis:** Das System liest die Module nun ein und stellt sie als neue, interaktive Tabs in der "Workspace-Ebene" (untere Tab-Leiste) zur Verfügung.

3. Das A.R.T.E.App-Panel.html (Simple App Store)

Das Modul `A.R.T.E.App-Panel.html` ist eine revolutionäre Sandbox-Umgebung innerhalb des Systems. Es fungiert als lokaler App Store, der es ermöglicht, Micro-Apps isoliert zu laden, zu verwalten und direkt zu programmieren.

Bedienung & App-Management

- **Externe Apps laden:** Ziehe HTML-App-Dateien per Drag & Drop in die dedizierte Store-Zone. Sie werden sofort in einer lokalen Datenbank (IndexedDB) gespeichert und als dynamischer Tab in der oberen Leiste installiert.
- **Tabs & Navigation:** Apps reihen sich nahtlos als Tabs auf (z. B. App 1, App 2). Beim Schließen einer App über das rote **X** rücken die verbleibenden Apps in der Nummerierung automatisch auf.
- **Auto-Heal I-Frame:** Das System zerstört beim Tab-Wechsel das alte I-Frame und generiert ein frisches, um "White-Screen"-Darstellungsfehler (Display-None-Death) zu verhindern.
- **Export:** Jede laufende App kann über den "Export"-Button als fertige, eigenständige `.html` - Datei gesichert werden.

Terminal Node (Apps selbst programmieren)

Klicke auf den Tab "Terminal Node", um den integrierten Code-Editor zu öffnen. Hier können eigene HTML-basierte Apps geschrieben und direkt in den Store kompiliert werden.

Wichtige Richtlinien für das Schreiben eigener Apps:

1. **HTML-Fokus:** Es wird ausschließlich HTML-basierter Code im Raw-Format akzeptiert.
 2. **Dimensionen:** Die App sollte sich fließend (fluid) anpassen, da das System versucht, sie automatisch in den rechten Arbeitsbereich (I-Frame) einzupassen.
 3. **Auto-i18n Injection:** Fehlt in deinem Code die Anbindung an den System-Sprach-Bus (`window.osBus.on('lang-changed')`), injiziert das Terminal diese Übersetzungslogik beim Speichern vollautomatisch in den Hintergrund deines Codes. Die App ist danach sofort systemkompatibel.
 4. **Speichern & Bestätigen:** Nach dem Schreiben muss der Code durch einen Klick auf " App Kompilieren & Speichern" bestätigt werden. Die App taucht danach sofort im Tab-Menü auf. Beim späteren Exportieren ist die injizierte System-Übersetzungslogik fest im Code verankert.
-

4. Übersicht der verfügbaren Module

Die folgenden Module können nach dem Systemstart geladen und genutzt werden:

-  **A.R.T.E.App-Panel.html**: Der lokale Simple App Store mit dynamischen Tabs, I-Frame-Isolierung und integriertem Terminal Node für die App-Entwicklung.
 -  **ARTE_Currency_Core.html**: Eine autarke Währungs-Umrechnungsmatrix mit Live-Satelliten-Uplink für aktuelle Kurse und einer integrierten Offline-Fallback-Datenbank für höchste Ausfallsicherheit.
 -  **ARTE_Sci_Calc.html**: Ein wissenschaftlicher Taschenrechner mit Kinetic UI und Safe-Eval-Routing. Berechnet komplexe mathematische Formeln (sin, cos, log, etc.) in einer streng isolierten Sandbox.
 -  **A.R.T.E. Quantum Canvas**: Eine hybride Raster/Vektor-Zeichenumgebung mit dynamischer Workspace-Expansion, Symmetrie-Matrix, Rotations-Werkzeugen und synchroner Export-Engine.
 -  **ARTE_Chronos_Matrix.html**: Ein sphärisches Time-Engine-Modul mit Kinetic UI, Hardware Canvas Rendering und globaler Zeitzone-Synchronisation samt persistentem Local-Storage.
 -  **Audio_Analyser-Pipeline.html (Acoustic Resonance Engine)**: Visualisiert Audio-Dateien in Echtzeit mit reaktiven Effekten (z. B. Cyberpunk Matrix, Nebula).
 -  **Image_Resizer_Matrix.html (High Fidelity Scaler)**: Ermöglicht präzises Skalieren von Bildern unter Beibehaltung der Proportionen samt verlustfreiem Export (PNG, JPG, WEBP).
 -  **Kinetic Telemetry&_Canvas_Engine.html**: Ein GPU-beschleunigtes Oszilloskop zur Echtzeit-Datenvisualisierung auf einem HTML5-Canvas.
 -  **Optical Contour&_Blueprint_Synthesizer.html**: Wandelt Bilder lokal per Kantenerkennung in detaillierte Blaupausen um.
 -  **Kinetic Capture & Imaging Engine**: Koppelt das System mit Video-Feeds für Snapshots und direkte Bildbearbeitung im Fokus-Raum.
 -  **Advanced Document Synthesis Matrix**: Lokaler WYSIWYG-Texteditor mit A4-Papiersimulation und PDF/HTML/MD/TXT-Export.
 -  **Universal Media Converter**: Transkodiert Medien offline im Browser (MP3, MP4, WebM, M4A).
 -  **Kinetic Video Engine**: Erlaubt das Laden von Videos für Echtzeit-Wiedergabe, Zuschnitte und Codec-Export.
 -  **Global Weather Radar / 48H Forecast**: Interaktive Wetterkarte mit Radar-Animationen.
 -  **Live Web Synthesis Matrix**: Browser-IDE für HTML/CSS/JS mit Live-Vorschau.
-

5. Entwicklungsrichtlinien für neue Module

Beim Entwickeln neuer A.R.T.E. Module müssen zwingend folgende Architektur- und Inhaltsstandards eingehalten werden, um die Systemstabilität zu gewährleisten:

- **Shadow DOM Isolation:** Jedes Modul wird vom Kernsystem aus Sicherheitsgründen in einen eigenen isolierten Shadow DOM geladen. Globale Abfragen wie `document.getElementById()` schlagen fehl. Es muss eine dynamische Root-Referenz gebildet werden (z. B. `const root = typeof shadowRoot !== 'undefined' ? shadowRoot : document;`).
- **Lokales i18n & Event Bus (osBus):** Module müssen vollständig zweisprachig (Deutsch/Englisch) arbeiten. Das Modul muss über den globalen Event-Bus auf Sprachwechsel lauschen (`window.osBus.on('lang-changed', ...)`), um die Oberfläche dynamisch ohne Neuladen zu aktualisieren.
- **Wissenschaftliche Präzision & Funktionalität:** Jegliche Berechnungen und Prozesse müssen mathematisch präzise, nachvollziehbar sowie physikalisch und chemisch konsistent aufgebaut sein. Unbelegte Annahmen sind zu vermeiden.
- **Universeller Nutzen & Strikte Neutralität:** Der Code und die Benutzeroberfläche müssen das Potenzial für einen breiten, allgemeinen Nutzen bieten. Die Einbindung jeglicher politischer Aussagen, Wertungen oder Tendenzen ist ausnahmslos untersagt.